

ELTE EÖTVÖS LORÁND UNIVERSITY

INTRODUCTION TO SIGNALR

Modern web application development in .NET

Lecture 11

Dr. Máté Cserép

mcserep@inf.elte.hu

https://mcserep.web.elte.hu/

REAL-TIME COMMUNICATION

Modern applications often need to update the UI without user interaction.

- Examples include live chat, multiplayer games, live dashboards, collaborative tools, etc.
- Traditional HTTP is insufficient for low-latency, bi-directional communication.
- This is where technologies like WebSocket and ASP.NET Core SignalR become essential.

WEBSOCKET

WebSocket is a protocol that provides a persistent, full-duplex communication channel over a single TCP connection.

- Unlike HTTP, which follows a request-response model, WebSockets allow **continuous, bidirectional communication between client and server**, including the server to push data to clients at any time.
- This significantly reduces overhead and improves responsiveness.
- It is ideal for scenarios requiring low latency and real-time updates.
- WebSocket supports text and binary data transfer as well.
- Uses the **`ws://`** and **`wss://`** protocol names in URLs.

WEBSOCKET HANDSHAKE

The WebSocket protocol begins with a standard HTTP request, upgraded to WebSocket via headers, e.g.:

Client Request:

```
GET /chat HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
```

Server Response:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
```

Once this handshake is done successfully, both sides can exchange data directly over TCP.

WEBSOCKET DATAFRAME

WebSocket communicates with *data frames*. Structure for each frame:

- **FIN** (1 bit): indicates whether this is the final fragment (WebSocket supports frame fragmentation)
- **Opcode** (4 bits): type of the frame (*text*, *binary*, *ping*, *pong*, *close*, etc.):
- **Mask** (1 bit): security feature to mask the payload with a random key
- **Payload Length** (7+ bits): length of the message
- **Payload Data**: actual content of the message

WEBSOCKET MESSAGE FLOW

The WebSocket Message Flow is the following:

1. Client sends handshake to upgrade HTTP → WebSocket
2. Server responds with 101 Switching Protocols
3. Connection established over TCP
    - Either side can send messages at any time (no need to wait)
    - Heartbeat via *ping/pong* ensures connection is alive
4. Sending a *close frame* gracefully terminates the connection

HTTP AND WEBSOCKET COMPARISON

Feature	HTTP	WebSocket
Communication Model	Request → Response	Bidirectional, event-based
Connection Type	Stateless (short-lived)	Stateful (long-lived)
Latency	High (new connections, headers per request)	Low (constant open connection after initial handshake)
Server Push Support	No native support (requires polling or SSE)	Yes (server can send data anytime)
Transport Protocol	HTTP/1.1 or HTTP/2 over TCP	TCP (after HTTP handshake upgrade)
Scalability	Well-supported by CDNs, caches, load balancers	Requires WebSocket-aware infrastructure
Security	HTTPS (HTTP over TLS)	WSS (WS over TLS)

WHAT IS ASP.NET SIGNALR?

ASP.NET Core SignalR is a library to easily add features based on real-time functionality (like live chat, dashboards, or notifications) to ASP.NET Core based web application.

Key features:

- Automatic transport negotiation
- Automatic connection management
- Group messaging
- Scalable backplane support (e.g., *Redis*, *Azure SignalR Service*)
- Strong client-side support across platforms

AUTOMATIC TRANSPORT NEGOTIATION

When WebSockets are unavailable (e.g., due to firewalls or proxies), SignalR falls back to:

- **Server-Sent Events (SSE):** Server pushes text-based updates to client (unidirectional). This is included in the HTML5 standard and supported by modern browsers.
- **Long Polling:** Client polls the server repeatedly for updates, simulating real-time.

SignalR hides these details and always chooses the best transport automatically.

SIGNALR CORE COMPONENTS

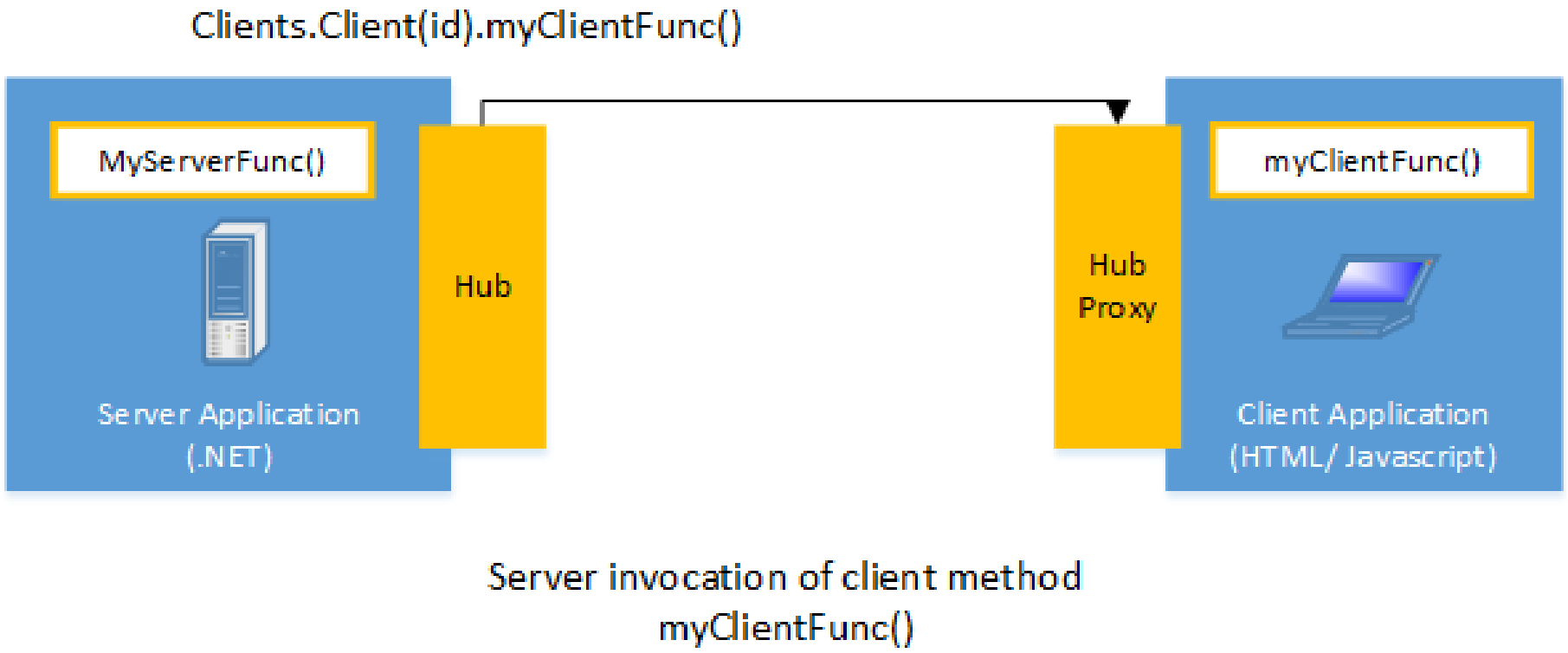
SignalR consists of three main elements:

1. **Hub:** The central point of communication on the server
2. **Clients:** Applications connected via WebSocket/HTTP
3. **Connection Protocol:** Handles message format and transport negotiation

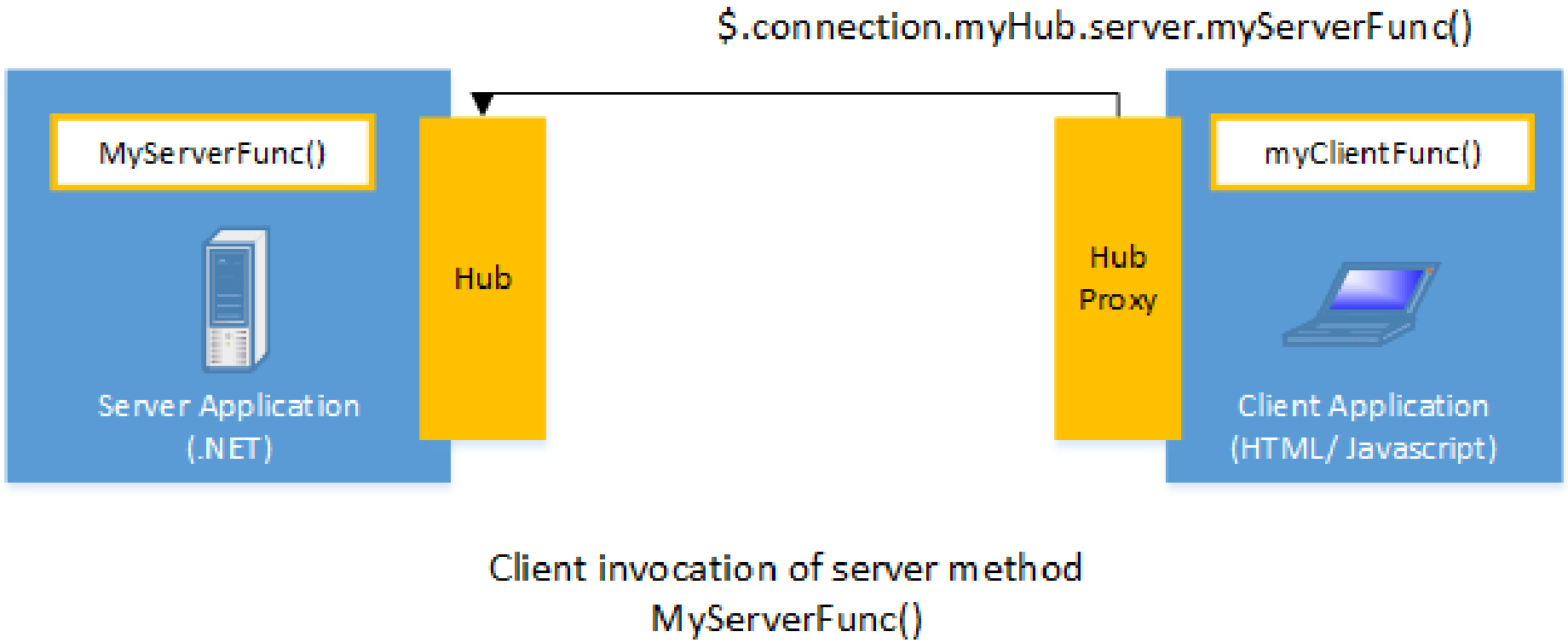
SignalR provides a high abstraction level API to communicate between hubs and clients through *remote procedure calls (RPC)*.

- Allows the server to call methods on connected clients and vice versa.

SERVER → CLIENT COMMUNICATION EXAMPLE



CLIENT → SERVER COMMUNICATION EXAMPLE



DEFINING A SIGNALR HUB

Creating a SignalR Hub in ASP.NET Core is done easily, inheriting from the **Hub** base class:

```
public class ChatHub : Hub
{
  public async Task SendMessage(string user, string message)
  {
    await Clients.All.SendAsync(
      "ReceiveMessage", user, message);
  }
}
```

STRONGLY TYPED HUBS

Beside *dynamic dispatching*, defining a contract between the hub and the client is also supported, using strongly typed hubs:

```
public interface IChatClient
{
  Task ReceiveMessage(string user, string message);
}

public class ChatHub : Hub<IChatClient>
{
  public async Task SendMessage(string user, string message) {
    await Clients.All.ReceiveMessage(user, message);
  }
}
```

CONTROLLING MESSAGE TARGETS

The Clients property hub has key properties and methods to manage message targets:

- **`Clients.All`**: calls a method on all connected clients
- **`Clients.Caller`**: calls a method on the client invoking the hub method
- **`Clients.Others`**: calls a method on all connected clients except the client that invoked the method

- **`Clients.Group(…)`**: calls a method on all connections in the group
- **`Clients.User(…)`**: calls a method on all connections associated with a specific user (e.g. to notify the user's other devices)

CONFIGURING SIGNALR ON THE SERVER

Adding SignalR support in a ASP.NET Core web app can be done through installing the **`Microsoft.AspNetCore.SignalR`** NuGet package.

- Then in the **`Program`** (or in the **`ConfigureServices()`** of **`Startup`**) enable it, by registering it to the service collection collection: **`builder.Services.AddSignalR();`**
- Define the new endpoints (typically one endpoint / hub), in the **`Program`** (or int **`Configure()`** of **`Startup`**):

```
var app = builder.Build();
…
app.MapHub<ChatHub>("/chatHub");
```

SIGNALR CLIENT SUPPORT

SignalR officially supports the following client platforms. Each client library is designed to work seamlessly with the server-side SignalR hubs.

- .NET
- JavaScript
- TypeScript
- Java
- Swift

Further programming environments have 3rd party support (e.g. Python).

.NET CLIENT

Install the **`Microsoft.AspNetCore.SignalR.Client`** NuGet package for your console, desktop or mobile client application.

- Then, it is ready to use, e.g. calling a method on a server hub:

```
var connection = new HubConnectionBuilder()
  .WithUrl("https://localhost:5001/chatHub")
  .WithAutomaticReconnect()
  .Build();

await connection.StartAsync();

await connection.InvokeAsync(
  "SendMessage", ".NET Client", "Hello from .NET!");
```

.NET CLIENT

- Or receiving incoming messages from the server:

```
var connection = new HubConnectionBuilder()
  .WithUrl("https://localhost:5001/chatHub")
  .WithAutomaticReconnect()
  .Build();

connection.On<string, string>(
  "ReceiveMessage", (user, message) => {
    Console.WriteLine($"{user}: {message}");
  }
);
await connection.StartAsync();
```

JAVASCRIPT CLIENT

Install the **@microsoft/signalr** NPM package, or load through a CDN.

- Then, it is ready to use, e.g. calling a method on a server hub:

```
<script src="~/lib/signalr/signalr.js"></script>
<script>
  const connection = new signalR.HubConnectionBuilder()
    .withUrl("/chatHub")
    .withAutomaticReconnect()
    .build();

  connection.start().then(() => {
    connection.invoke("SendMessage",
                      "JS Client", "Hi from JS!");
  });
</script>
```

JAVASCRIPT CLIENT

- Or receiving incoming messages from the server:

```
<script src="~/lib/signalr/signalr.js"></script>
<script>
  const connection = new signalR.HubConnectionBuilder()
    .withUrl("/chatHub")
    .withAutomaticReconnect()
    .build();

  connection.on("ReceiveMessage", (user, message) => {
    console.log(`${user}: ${message}`);
  });

  connection.start();
</script>
```

TYPESCRIPT CLIENT

Install the **`@microsoft/signalr`** NPM package, or load through a CDN.

- Then, it is ready to use, e.g. calling a method on a server hub:

```
import * as signalR from "@microsoft/signalr";

const connection = new signalR.HubConnectionBuilder()
    .withUrl("/chatHub")
    .withAutomaticReconnect()
    .build();

await connection.start();
await connection.invoke(
  "SendMessage", "TS Client", "Message from TypeScript");
```

TYPESCRIPT CLIENT

- Or receiving incoming messages from the server:

```
import * as signalR from "@microsoft/signalr";

const connection = new signalR.HubConnectionBuilder()
    .withUrl("/chatHub")
    .withAutomaticReconnect()
    .build();

connection.on(
  "ReeiveMessage", (user: string, message: string) => {
    console.log(`${user}: ${message}`);
});

await connection.start();
```

JAVA CLIENT

Integrating SignalR differs on the build system used in the Java project.

- Using Gradle, add the following line to the dependencies section of your *build.gradle* file:
  `implementation 'com.microsoft.signalr:signalr:8.0.15'`
- Using Maven, add the following lines inside the <dependencies> element of your *pom.xml* file:

```
<dependency>
    <groupId>com.microsoft.signalr</groupId>
    <artifactId>signalr</artifactId>
    <version>8.0.15</version>
</dependency>
```

JAVA CLIENT

- Then, it is ready to use, e.g. calling a method on a server hub and listening for server message:

```
HubConnection hubConnection = HubConnectionBuilder
   .create("https://localhost:5001/chatHub")
   .build();

hubConnection.on("ReceiveMessage", (user, message) -> {
   System.out.println(user + ": " + message);
}, String.class, String.class);

hubConnection.start().blockingAwait()
hubConnection.send("SendMessage",
                   "Java User", "Hello from Java!");
```